

Nome:

Nota:

QUADRO DE RESPOSTAS

Q.01		Q.02		Q.03		Q.04		Q.05	
Q.06		Q.07		Q.08		Q.09		Q.10	
Q.11		Q.12		Q.13		Q.14		Q.15	
Q.16		Q.17		Q.18		Q.19		Q.20	

CENÁRIO

Considere o sistema abaixo, usado para representar uma eleição de representantes do IFPB.

Analise atentamente as classes e seus métodos. Todas as respostas da prova devem ser derivadas exclusivamente do código apresentado.

Candidato.java

```
1 public class Candidato {
2
3     private int numero;
4     private String nome;
5
6     public Candidato(int numero, String nome) {
7         this.numero = numero;
8         this.nome = nome;
9     }
10
11     public int getNumero() {
12         return numero;
13     }
14
15     public String getNome() {
16         return nome;
17     }
18 }
```

Voto.java

```
1 public class Voto {
2
3     private Candidato candidato;
4
5     public Voto(Candidato candidato) {
6         this.candidato = candidato;
7     }
8
9     public boolean ehPara(Candidato cand) {
10         return this.candidato == cand;
11     }
12 }
```

Eleitor.java

```
1 public class Eleitor {
2
3     private String identificacao;
4     private boolean votou;
5
6     public Eleitor(String identificacao) {
7         this.identificacao = identificacao;
8         this.votou = false;
9     }
10
11     public String getIdentificacao() {
12         return identificacao;
13     }
14
15     public boolean jaVotou() {
16         return votou;
17     }
18
19     public void registrarVoto() {
20         if (!votou) {
21             votou = true;
22         }
23     }
24 }
```

QUESTÕES

Q1. (5 pontos) Observando as classes do sistema, qual alternativa descreve melhor a distribuição de responsabilidades adotada?

- A) **Eleitor** controla o registro dos votos, enquanto **Eleicao** apenas mantém candidatos cadastrados.
- B) **Voto** decide se uma votação pode ocorrer, enquanto **Candidato** controla os votos que recebe.
- C) **Eleicao** coordena a votação, **Eleitor** mantém seu próprio estado e **Voto** registra o candidato escolhido.
- D) **Candidato** coordena a votação, **Eleitor** registra o candidato escolhido e **Eleicao** mantém apenas os votos.
- E) **Eleicao** concentra o estado dos eleitores, candidatos e votos, deixando os demais objetos apenas como dados.

Q2. (5 pontos) Suponha que a classe **Eleicao oferecesse o método abaixo:**

```
public List<Voto> getVotos() {  
    return votos;  
}
```

Qual seria a principal consequência dessa decisão para o modelo apresentado?

- A) O cliente poderia consultar os votos, mas continuaria impedido de alterar a coleção mantida pela eleição.
- B) O cliente poderia alterar diretamente a coleção interna, contornando operações e regras controladas por **Eleicao**.
- C) A coleção seria copiada a cada chamada, fazendo com que alterações externas não afetassem a eleição.
- D) Os objetos **Voto** deixariam de manter referências para candidatos e passariam a pertencer ao cliente.
- E) A eleição deixaria de conseguir percorrer seus votos, pois a lista passaria a ser controlada externamente.

Q3. (5 pontos) Um aluno propõe acrescentar o método abaixo à classe **Eleitor:**

```
public void setVotou(boolean votou) {  
    this.votou = votou;  
}
```

Considerando o restante do modelo, qual análise é mais adequada?

- A) O método é adequado porque todo campo privado deve possuir uma operação pública para escrita.
- B) O método é adequado porque permite que **Eleicao** altere o estado do eleitor sem chamar **registrarVoto()**.

C) O método enfraquece o controle do estado, pois permite alterações que não correspondem necessariamente a uma votação realizada.

D) O método não produz diferença porque campos **boolean** não podem ser modificados depois do construtor.

E) O método é necessário porque **registrarVoto()** não consegue alterar um campo declarado como **private**.

Q4. (5 pontos) A classe **Voto mantém uma referência para **Candidato**, mas **Candidato** não mantém uma lista dos votos recebidos. O que melhor explica essa decisão no modelo atual?**

A) A relação inversa é proibida em Java quando uma das classes já mantém referência para a outra.

B) **Candidato** não precisa alcançar os votos para cumprir as responsabilidades que lhe foram atribuídas.

C) Uma relação entre dois objetos deve sempre existir em apenas uma direção para evitar referências duplicadas.

D) **Voto** possui uma cópia completa do candidato e, por isso, não seria possível manter a relação inversa.

E) A lista de votos só poderia existir em **Candidato** se a classe também mantivesse referência para **Eleicao**.

Q5. (5 pontos) **Eleicao mantém candidatos cadastrados e votos registrados. Considerando como esses objetos participam do modelo, qual interpretação é mais adequada?**

A) Candidatos e votos possuem o mesmo papel estrutural, pois ambos são armazenados em listas mantidas por **Eleicao**.

B) Candidatos formam partes estruturais da eleição, enquanto votos existem independentemente e apenas colaboram com ela.

C) Votos funcionam como partes estruturais mantidas pela eleição, enquanto candidatos podem existir independentemente e se associar a ela.

D) Candidatos e votos apenas colaboram temporariamente com a eleição, pois uma coleção não estabelece relação entre objetos.

E) Votos e candidatos só poderiam fazer parte da estrutura da eleição se também mantivessem uma referência para **Eleicao**.

Eleicao.java

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Eleicao {
5
6     private String nome;
7     private List<Candidato> candidatos;
8     private List<Voto> votos;
9     private boolean encerrada;
10
11     public Eleicao(String nome) {
12         this.nome = nome;
13         candidatos = new ArrayList<>();
14         votos = new ArrayList<>();
15         encerrada = false;
16     }
17
18     public void cadastrarCandidato(Candidato candidato) {
19         if (!encerrada && !candidatoCadastrado(candidato)) {
20             candidatos.add(candidato);
21         }
22     }
23
24     public void votar(Eleitor eleitor, Candidato candidato) {
25         if (!encerrada
26             && !eleitor.jaVotou()
27             && candidatoCadastrado(candidato)) {
28
29             Voto voto = new Voto(candidato);
30             votos.add(voto);
31             eleitor.registrarVoto();
32         }
33     }
34
35     public void retirarCandidato(Candidato candidato) {
36         if (!encerrada && !possuiVotoPara(candidato)) {
37             for (int i = 0; i < candidatos.size(); i++) {
38                 if (candidatos.get(i) == candidato) {
39                     candidatos.remove(i);
40                     return;
41                 }
42             }
43         }
44     }
45
46     public void encerrar() {
47         encerrada = true;
48     }
49 }
```

```
50     public int quantidadeCandidatos() {
51         return candidatos.size();
52     }
53
54     public int quantidadeVotos() {
55         return votos.size();
56     }
57
58     public int totalVotosDe(Candidato candidato) {
59         int total = 0;
60
61         for (Voto voto : votos) {
62             if (voto.ehPara(candidato)) {
63                 total++;
64             }
65         }
66
67         return total;
68     }
69
70     public boolean isEncerrada() {
71         return encerrada;
72     }
73
74     private boolean candidatoCadastrado(Candidato candidato) {
75         for (Candidato atual : candidatos) {
76             if (atual == candidato) {
77                 return true;
78             }
79         }
80
81         return false;
82     }
83
84     private boolean possuiVotoPara(Candidato candidato) {
85         for (Voto voto : votos) {
86             if (voto.ehPara(candidato)) {
87                 return true;
88             }
89         }
90
91         return false;
92     }
93 }
```

QUESTÕES – CONTINUAÇÃO

Questões 6 a 20 – Leitura de código (75 pontos)

Cada trecho abaixo representa uma execução independente. Os objetos criados em um trecho não existem nos demais.

Para cada questão, escreva exatamente o conteúdo exibido pela impressão identificada com seu número. Cada impressão corresponde a uma questão e vale 5 pontos. Não inclua aspas nas respostas textuais.

Trecho I – Questões 6 a 8

```
1 public class Main {
2     public static void main(String[] args) {
3         Candidato ana = new Candidato(15, "Ana");
4         Candidato referencia = ana;
5
6         Eleicao eleicao = new Eleicao("Conselho");
7         eleicao.cadastrarCandidato(referencia);
8
9         referencia = new Candidato(20, "Bia");
10        eleicao.retirarCandidato(referencia);
11
12        System.out.println(eleicao.quantidadeCandidatos()); // Q6
13
14        Candidato alias = ana;
15        eleicao.cadastrarCandidato(alias);
16
17        Candidato outraAna = new Candidato(15, "Ana");
18        eleicao.cadastrarCandidato(outraAna);
19
20        System.out.println(eleicao.quantidadeCandidatos()); // Q7
21
22        Eleitor titular = new Eleitor("E1");
23        Eleitor apoio = titular;
24
25        eleicao.votar(apoio, outraAna);
26
27        System.out.println(titular.jaVotou()); // Q8
28    }
29 }
```

Trecho II – Questões 9 a 11

```
1 public class Main {
2     public static void main(String[] args) {
3         Candidato ana1 = new Candidato(15, "Ana");
4         Candidato ana2 = new Candidato(15, "Ana");
5         Candidato caio = new Candidato(30, "Caio");
6
7         Eleicao eleicao = new Eleicao("Conselho");
8
9         eleicao.cadastrarCandidato(ana1);
10        eleicao.cadastrarCandidato(ana2);
11
12        Eleitor e1 = new Eleitor("X");
13        Eleitor e2 = new Eleitor("X");
14    }
```

```

15     eleicao.votar(e1, ana1);
16
17     System.out.println(e2.jaVotou());           // Q9
18
19     Eleitor e3 = new Eleitor("E3");
20
21     eleicao.votar(e3, caio);
22     eleicao.cadastrarCandidato(caio);
23     eleicao.votar(e3, caio);
24
25     System.out.println(eleicao.totalVotosDe(caio)); // Q10
26
27     eleicao.votar(e2, ana2);
28     eleicao.votar(new Eleitor("E4"), ana2);
29     eleicao.votar(new Eleitor("E5"), ana1);
30
31     System.out.println(eleicao.totalVotosDe(ana1)); // Q11
32 }
33 }

```

Trecho III – Questões 12 a 14

```

1 public class Main {
2     public static void main(String[] args) {
3         Candidato ana1 = new Candidato(15, "Ana");
4         Candidato ana2 = new Candidato(15, "Ana");
5         Candidato bia = new Candidato(20, "Bia");
6
7         Eleicao eleicao = new Eleicao("Conselho");
8
9         eleicao.cadastrarCandidato(ana1);
10        eleicao.cadastrarCandidato(ana2);
11        eleicao.cadastrarCandidato(bia);
12
13        eleicao.retirarCandidato(ana1);
14
15        System.out.println(eleicao.quantidadeCandidatos()); // Q12
16
17        Eleicao outraEleicao = new Eleicao("Representacao");
18        Eleitor outroEleitor = new Eleitor("E2");
19
20        outraEleicao.cadastrarCandidato(ana1);
21        outraEleicao.votar(outroEleitor, ana1);
22
23        System.out.println(outraEleicao.quantidadeVotos()); // Q13
24
25        eleicao.votar(new Eleitor("E1"), ana2);
26
27        eleicao.retirarCandidato(ana2);
28        eleicao.retirarCandidato(bia);
29
30        System.out.println(eleicao.quantidadeCandidatos()); // Q14
31    }
32 }

```

Trecho IV – Questões 15 a 17

```
1 public class Main {
2     public static void main(String[] args) {
3         Candidato ana = new Candidato(15, "Ana");
4         Candidato bia = new Candidato(20, "Bia");
5
6         Eleitor compartilhado = new Eleitor("E1");
7         Eleitor livre = new Eleitor("E2");
8
9         Eleicao manha = new Eleicao("Manha");
10        Eleicao tarde = new Eleicao("Tarde");
11
12        manha.cadastrarCandidato(ana);
13        manha.cadastrarCandidato(bia);
14
15        tarde.cadastrarCandidato(ana);
16        tarde.cadastrarCandidato(bia);
17
18        manha.votar(compartilhado, ana);
19
20        tarde.votar(compartilhado, bia);
21        tarde.votar(livre, bia);
22
23        System.out.println(tarde.quantidadeVotos());           // Q15
24
25        manha.retirarCandidato(bia);
26        tarde.retirarCandidato(bia);
27
28        System.out.println(tarde.quantidadeCandidatos());       // Q16
29
30        Eleicao atual = manha;
31        atual.encerrar();
32
33        atual = new Eleicao("Noite");
34        atual.cadastrarCandidato(bia);
35        atual.encerrar();
36        atual.cadastrarCandidato(ana);
37
38        System.out.println(atual.quantidadeCandidatos());       // Q17
39    }
40 }
```

Trecho V – Questões 18 a 20

```
1 public class Main {
2     public static void main(String[] args) {
3         Candidato ana = new Candidato(15, "Ana");
4         Candidato bia = new Candidato(20, "Bia");
5
6         Candidato escolha = ana;
7         Candidato memoria = escolha;
8
9         Eleitor primeiro = new Eleitor("E1");
10        Eleitor segundo = new Eleitor("E2");
11    }
```

```
12     Eleicao principal = new Eleicao("Principal");
13
14     principal.cadastrarCandidato(ana);
15     principal.cadastrarCandidato(bia);
16
17     principal.votar(primeiro, escolha);
18
19     escolha = bia;
20     principal.votar(segundo, escolha);
21
22     escolha = memoria;
23     principal.votar(new Eleitor("E3"), escolha);
24
25     memoria = bia;
26
27     System.out.println(principal.totalVotosDe(memoria));           // Q18
28
29     Eleitor eleitor = new Eleitor("E2");
30     Eleicao ordem = new Eleicao("Ordem");
31
32     ordem.cadastrarCandidato(ana);
33     ordem.retirarCandidato(ana);
34
35     ordem.votar(eleitor, ana);
36
37     ordem.cadastrarCandidato(ana);
38     ordem.votar(eleitor, ana);
39
40     ordem.retirarCandidato(ana);
41
42     System.out.println(ordem.quantidadeCandidatos());           // Q19
43
44     Candidato caio = new Candidato(30, "Caio");
45     Eleitor e3 = new Eleitor("E3");
46
47     Eleicao finalEleicao = new Eleicao("Final");
48
49     finalEleicao.cadastrarCandidato(ana);
50     finalEleicao.cadastrarCandidato(bia);
51
52     finalEleicao.votar(new Eleitor("E4"), ana);
53
54     finalEleicao.retirarCandidato(bia);
55     finalEleicao.cadastrarCandidato(caio);
56
57     finalEleicao.encerrar();
58
59     finalEleicao.retirarCandidato(caio);
60     finalEleicao.cadastrarCandidato(bia);
61     finalEleicao.votar(e3, caio);
62
63     System.out.println(finalEleicao.quantidadeCandidatos()); // Q20
64 }
65 }
```